LLMOps Assignment 2: API Cost, Latency, and Prompt Version Management (Colab version)

Scenario: EduAssist AI academic Q&A assistant | **Provider:** Ollama (local open-source model, OpenAI-compatible endpoint) running on Google Colab

OpenAI credits were exhausted (`insufficient_quota`, HTTP 429), so, as allowed in section 7 of the brief, this notebook uses a locally hosted open-source model. No API key is needed.

How to run:

1. Runtime > Change runtime type > **T4 GPU** (free). If you only have CPU, change `MODEL` below to `qwen2.5:3b`.
2. Run all cells top to bottom.
3. The last cell zips `mlflow.db`, `mlartifacts` and the CSVs for download. View the MLflow UI on your own laptop (command in the last cell) and take screenshots there.

Setup: install packages and Ollama

```
!pip -q install "mlflow>=3.0" openai pandas
!apt-get -qq install -y zstd pciutils > /dev/null
!curl -fsSL https://ollama.com/install.sh | sh
```

```
>>> Cleaning up old version at /usr/local/lib/ollama
>>> Installing ollama to /usr/local
>>> Downloading ollama-linux-amd64.tar.zst
######################################################################## 100.0%
>>> Adding ollama user to video group...
>>> Adding current user to ollama group...
>>> Creating ollama systemd service...
WARNING: systemd is not running
>>> NVIDIA GPU installed.
>>> The Ollama API is now available at 127.0.0.1:11434.
>>> Install complete. Run "ollama" from the command line.
```

Start the Ollama server and download the model

```
import subprocess, time, os
MODEL = "llama3.1:8b"   # T4 GPU. On a CPU-only runtime use "qwen2.5:3b" instead.

server = subprocess.Popen(["ollama", "serve"], stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)
time.sleep(8)
!ollama pull {MODEL}
```

Configuration

```
import os, sys, time, json, difflib, datetime, subprocess
import pandas as pd
import mlflow
from openai import OpenAI
from IPython.display import display, Markdown

# ---- Configuration (document these in the report) ----
PROVIDER = "Ollama (local open-source model) on Google Colab"
MODEL = "llama3.1:8b"          # must match the model pulled above
API_VERSION = "OpenAI-compatible Chat Completions API served by Ollama (openai Python SDK v1+)"
CURRENCY = "USD"

# Ollama is free, so cost is ESTIMATED from a published hosted price for the same model family.
# STEP 1: open a provider's pricing page for Llama 3.1 8B (Groq, Together, DeepInfra, ...).
# STEP 2: fill the four lines below with what the page says TODAY.
# STEP 3: set PRICES_VERIFIED = True. The notebook refuses to run until you do.
PRICING_SOURCE = "DeepInfra (deepinfra/fp8) via OpenRouter, meta-llama/llama-3.1-8b-instruct, listed price, https://openrout
PRICING_DATE = "2026-11-04"
INPUT_PRICE_PER_M = 0.02
OUTPUT_PRICE_PER_M = 0.04
PRICES_VERIFIED = True

assert PRICES_VERIFIED and INPUT_PRICE_PER_M is not None and OUTPUT_PRICE_PER_M is not None \
    and "TODO" not in PRICING_SOURCE + PRICING_DATE, \
```

```
    "Fill PRICING_SOURCE, PRICING_DATE and both prices from the provider's pricing page, then set PRICES_VERIFIED = True."

MONTHLY_REQUESTS = 10_000
TEMPERATURE = 0
EXPERIMENT_NAME = "eduassist_llmops_assignment"
PAUSE_BETWEEN_CALLS = 0.5      # seconds

# Hardware actually used (goes into the report and README)
try:
    GPU_NAME = subprocess.run(["nvidia-smi", "--query-gpu=name", "--format=csv,noheader"],
                              capture_output=True, text=True, timeout=10).stdout.strip() or "no GPU detected"
except Exception:
    GPU_NAME = "no GPU detected"
print("Hardware:", GPU_NAME)

# Local endpoint: the api_key value is ignored by Ollama, so there is no secret to hide.
client = OpenAI(base_url="http://localhost:11434/v1", api_key="ollama", timeout=300, max_retries=0)

mlflow.set_tracking_uri("sqlite:///mlflow.db")
mlflow.set_experiment(EXPERIMENT_NAME)

# Warm-up call: the first request loads the model into memory, so keep it out of the timed runs.
_t = client.chat.completions.create(model=MODEL, messages=[{"role": "user", "content": "Say hi."}])
print(_t.choices[0].message.content, "| usage:", _t.usage)
print(subprocess.run(["ollama", "ps"], capture_output=True, text=True).stdout)   # PROCESSOR column shows GPU/CPU use
```

```
Hardware: Tesla T4
Hi! How's your day going so far? | usage: CompletionUsage(completion_tokens=11, prompt_tokens=13, total_tokens=24, completic
NAME           ID              SIZE      PROCESSOR    CONTEXT    RUNNER      UNTIL
llama3.1:8b    46e0c10c039e    5.3 GB    100% GPU     4096       llamacpp    4 minutes from now
```

Questions and prompt templates

```
QUESTIONS = [
    {"qid": "Q1", "text": "What is the difference between supervised and unsupervised learning?"},
    {"qid": "Q2", "text": "Explain what overfitting is and how to reduce it."},
    {"qid": "Q3", "text": "What is the time complexity of binary search and why?"},
    {"qid": "Q4", "text": "What is the difference between a process and a thread in an operating system?"},
    {"qid": "Q5", "text": "Explain the bias-variance tradeoff."},
]

PROMPT_NAME = "eduassist_qa_prompt"
PROMPTS = {
 "v1": ("Answer the following student question clearly.\n\nQuestion: {{question}}",
        "v1 Basic: plain instruction, no role, no format constraints"),
 "v2": ("You are an academic assistant.\nAnswer the following student question in a clear, concise, and accurate manner.\n"
        "Use simple language suitable for undergraduate students.\n\nQuestion: {{question}}",
        "v2 Structured: adds role, tone, and audience constraints"),
 "v3": ("You are an academic assistant.\nAnswer the student question and return ONLY valid JSON.\n"
        "Do not include markdown, explanations outside JSON, or code fences.\n\nUse this exact JSON schema:\n"
        "{\n  \"answer\": \"string\",\n  \"confidence\": \"high | medium | low\",\n  \"key_points\": [\"string\"]\n}\n\n"
        "Question: {{question}}",
        "v3 JSON Output: strict JSON schema for downstream parsing"),
}
```

Task 1: Call the LLM and measure latency

```
def render(template, question):
    return template.replace("{{question}}", question)

def call_llm(prompt_text, retries=4):
    """Returns (text, (in_tok, out_tok, total_tok), status, start_iso, end_iso, latency_s).
    Latency is the duration of the final (successful) attempt only; failures are printed, not hidden."""
    last_err = None
    for attempt in range(retries):
        start_iso = datetime.datetime.now().isoformat(timespec="milliseconds")
        t0 = time.perf_counter()
        try:
            r = client.chat.completions.create(
                model=MODEL, temperature=TEMPERATURE,
                messages=[{"role": "user", "content": prompt_text}])
            latency = time.perf_counter() - t0
            end_iso = datetime.datetime.now().isoformat(timespec="milliseconds")
            u = r.usage
            return (r.choices[0].message.content,
```

```
                    (u.prompt_tokens, u.completion_tokens, u.total_tokens),
                    "success", start_iso, end_iso, latency)
        except Exception as e:
            last_err = e
            print(f"  attempt {attempt+1}/{retries} failed: {type(e).__name__}: {e}")
            if attempt < retries - 1:
                time.sleep(min(5 * 2 ** attempt, 60))
    end_iso = datetime.datetime.now().isoformat(timespec="milliseconds")
    return f"ERROR: {last_err}", (0, 0, 0), "error", start_iso, end_iso, time.perf_counter() - t0

def cost_of(inp, out):
    ic = inp / 1_000_000 * INPUT_PRICE_PER_M
    oc = out / 1_000_000 * OUTPUT_PRICE_PER_M
    return ic, oc, ic + oc

def run_version(version, template):
    rows = []
    for q in QUESTIONS:
        text, (inp, out, tot), status, s, e, lat = call_llm(render(template, q["text"]))
        ic, oc, tc = cost_of(inp, out)
        rows.append(dict(question_id=q["qid"], prompt_version=version, model=MODEL, start_time=s, end_time=e,
                         latency_seconds=lat, status=status, input_tokens=inp, output_tokens=out, total_tokens=tot,
                         input_cost=ic, output_cost=oc, total_cost=tc, response=text))
        time.sleep(PAUSE_BETWEEN_CALLS)
    return pd.DataFrame(rows)

# Tasks 1 and 2 use prompt v2 across the five questions
df_t1 = run_version("v2", PROMPTS["v2"][0])
assert (df_t1.status == "success").all(), "Some calls failed - read the errors printed above and fix before continuing."
df_t1[["question_id", "prompt_version", "model", "start_time", "end_time", "latency_seconds", "status"]]
```

	question_id	prompt_version	model	start_time	end_time	latency_seconds	status
0	Q1	v2	llama3.1:8b	2026-11-04T12:53:09.009	2026-11-04T12:53:16.731	7.721972	success
1	Q2	v2	llama3.1:8b	2026-11-04T12:53:17.233	2026-11-04T12:53:27.862	10.628457	success
2	Q3	v2	llama3.1:8b	2026-11-04T12:53:28.362	2026-11-04T12:53:40.942	12.580154	success
3	Q4	v2	llama3.1:8b	2026-11-04T12:53:41.442	2026-11-04T12:53:49.852	8.409818	success
4	Q5	v2	llama3.1:8b	2026-11-04T12:53:50.352	2026-11-04T12:54:03.609	13.256407	success

```
lat = df_t1["latency_seconds"]
summary = pd.DataFrame({"mean": [lat.mean()], "median": [lat.median()], "min": [lat.min()],
                        "max": [lat.max()], "std": [lat.std(ddof=1)]}).round(3)
display(summary)
slowest = df_t1.loc[lat.idxmax()]
print("Slowest request:", slowest.question_id, f"({slowest.latency_seconds:.2f}s, {slowest.output_tokens} output tokens)")
```

	mean	median	min	max	std
0	10.519	10.628	7.722	13.256	2.451

```
Slowest request: Q5 (13.26s, 426 output tokens)
```

Task 1 interpretation (generated from your measured latencies)

```
# Interpretation text is generated from YOUR measured numbers, so it always matches the tables above.
lat = df_t1["latency_seconds"]
i_max = lat.idxmax()
sl = df_t1.loc[i_max]
rest = df_t1.drop(i_max)
r = lat.corr(df_t1["output_tokens"])
tok_rate = df_t1["output_tokens"].sum() / lat.sum()

INTERP_T1 = (
    f"The slowest request was {sl.question_id} at {sl.latency_seconds:.2f} s with {sl.output_tokens} output tokens, "
    f"while the other four averaged {rest.latency_seconds.mean():.2f} s and {rest.output_tokens.mean():.0f} output tokens. "
    f"Latency and output tokens are strongly related across the five requests (r = {r:.2f}); the model produced about "
    f"{tok_rate:.0f} tokens per second, and tokens are generated one after another, so longer answers take longer. "
    f"There is no internet round trip ({GPU_NAME}, local server); other possible reasons are prompt length "
    f"({df_t1.input_tokens.min()} to {df_t1.input_tokens.max()} input tokens) and GPU scheduling. Five samples are only inc
)
display(Markdown("**Interpretation:** " + INTERP_T1))
```

Interpretation: The slowest request was Q5 at 13.26 s with 426 output tokens, while the other four averaged 9.84 s and 342 output tokens. Latency and output tokens are strongly related across the five requests (r = 0.98); the model produced about 34 tokens per second, and tokens are generated one after another, so longer answers take longer. There is no internet round trip (Tesla T4, local server); other possible reasons are prompt length (49 to 56 input tokens) and GPU scheduling. Five samples are only indicative.

Task 2: Token usage and cost

```
cols = ["question_id", "input_tokens", "output_tokens", "total_tokens", "input_cost", "output_cost", "total_cost"]
display(df_t1[cols])
avg_cost = df_t1["total_cost"].mean()
monthly = avg_cost * MONTHLY_REQUESTS
print(f"Average cost/request: {avg_cost:.6f} {CURRENCY}")
print(f"Projected monthly cost for {MONTHLY_REQUESTS:,} requests: {monthly:.2f} {CURRENCY}")
print(f"Provider={PROVIDER} | Model={MODEL} | API={API_VERSION} | Pricing source={PRICING_SOURCE} | Pricing date={PRICING_D
df_t1.drop(columns=["response"]).to_csv("latency_tokens_cost.csv", index=False)
```

	question_id	input_tokens	output_tokens	total_tokens	input_cost	output_cost	total_cost
0	Q1	53	281	334	1.060000e-06	0.000011	0.000012
1	Q2	53	376	429	1.060000e-06	0.000015	0.000016
2	Q3	52	432	484	1.040000e-06	0.000017	0.000018
3	Q4	56	278	334	1.120000e-06	0.000011	0.000012
4	Q5	49	426	475	9.800000e-07	0.000017	0.000018

```
Average cost/request: 0.000015 USD
Projected monthly cost for 10,000 requests: 0.15 USD
Provider=Ollama (local open-source model) on Google Colab | Model=llama3.1:8b | API=OpenAI-compatible Chat Completions API s
```

Task 2 cost-optimization recommendation (generated from your measured costs)

```
# Cost-optimization recommendation, quantified from your measured averages.
CAP = 200   # proposed max_tokens / "answer in at most ~150 words" limit
out_share = df_t1["output_cost"].sum() / df_t1["total_cost"].sum() * 100
ratio = OUTPUT_PRICE_PER_M / INPUT_PRICE_PER_M
capped = df_t1["output_tokens"].clip(upper=CAP)
new_cost = (df_t1["input_tokens"] / 1e6 * INPUT_PRICE_PER_M + capped / 1e6 * OUTPUT_PRICE_PER_M).mean()
saving_pct = (1 - new_cost / avg_cost) * 100
saving_month = (avg_cost - new_cost) * MONTHLY_REQUESTS
new_out = capped.mean()

REC_T2 = (
    f"Shorten responses. Output tokens made up {out_share:.0f}% of the measured cost because they are "
    f"priced {ratio:.1f} times higher than input tokens. Asking for concise answers and setting max_tokens to {CAP} would '
    f"cut average output from {df_t1.output_tokens.mean():.0f} to about {new_out:.0f} tokens, lowering the average cost per
    f"request from {avg_cost:.8f} to {new_cost:.8f} {CURRENCY} ({saving_pct:.0f}% less, about {saving_month:.4f} {CURRENCY}
    f"per {MONTHLY_REQUESTS:,} requests). Shorter output also reduces latency; check answer quality first."
)
display(Markdown("**Cost-optimization recommendation:** " + REC_T2))
```

Cost-optimization recommendation: Shorten responses. Output tokens made up 93% of the measured cost because they are priced 2.0 times higher than input tokens. Asking for concise answers and setting max_tokens to 200 would cut average output from 359 to about 200 tokens, lowering the average cost per request from 0.00001540 to 0.00000905 USD (41% less, about 0.0634 USD per 10,000 requests). Shorter output also reduces latency;

Task 3: Prompt registry, evaluation, and promotion in MLflow

```
registered = {}
for v, (tmpl, msg) in PROMPTS.items():
    p = mlflow.genai.register_prompt(name=PROMPT_NAME, template=tmpl, commit_message=msg)
    registered[v] = p.version
    print(v, "-> registry version", p.version)

def show_diff(a, b):
    ta, tb = PROMPTS[a][0].splitlines(), PROMPTS[b][0].splitlines()
    print("\n".join(difflib.unified_diff(ta, tb, f"prompt {a}", f"prompt {b}", lineterm="")))
show_diff("v1", "v2"); print("-" * 60); show_diff("v2", "v3")
```

```
v1 -> registry version 1
v2 -> registry version 2
v3 -> registry version 3
--- prompt v1
+++ prompt v2
```

```
@@ -1,3 +1,5 @@
-Answer the following student question clearly.
+You are an academic assistant.
+Answer the following student question in a clear, concise, and accurate manner.
+Use simple language suitable for undergraduate students.

 Question: {{question}}
------------------------------------------------------------
--- prompt v2
+++ prompt v3
@@ -1,5 +1,12 @@
 You are an academic assistant.
-Answer the following student question in a clear, concise, and accurate manner.
-Use simple language suitable for undergraduate students.
+Answer the student question and return ONLY valid JSON.
+Do not include markdown, explanations outside JSON, or code fences.
+
+Use this exact JSON schema:
+{
+  "answer": "string",
+  "confidence": "high | medium | low",
+  "key_points": ["string"]
+}

 Question: {{question}}
```

```
def is_valid_json(text):
    """Valid = parses as JSON with exactly the required keys and types (tolerates a markdown code fence)."""
    t = text.strip()
    if t.startswith("```"):
        t = t.strip("`")
        t = t[4:] if t.lower().startswith("json") else t
    try:
        d = json.loads(t)
    except Exception:
        return False
    return (isinstance(d, dict) and set(d.keys()) == {"answer", "confidence", "key_points"}
            and isinstance(d["answer"], str) and d["confidence"] in {"high", "medium", "low"}
            and isinstance(d["key_points"], list) and all(isinstance(k, str) for k in d["key_points"]))

all_rows, results = [], []
for v in PROMPTS:
    prompt = mlflow.genai.load_prompt(f"prompts:/{PROMPT_NAME}/{registered[v]}")
    df = run_version(v, prompt.template)
    df["valid_json"] = df["response"].apply(is_valid_json)
    all_rows.append(df)
    m = dict(avg_latency_seconds=df.latency_seconds.mean(), avg_input_tokens=df.input_tokens.mean(),
             avg_output_tokens=df.output_tokens.mean(), avg_cost_per_request=df.total_cost.mean(),
             valid_json=df.valid_json.mean())
    with mlflow.start_run(run_name=f"prompt_{v}"):
        mlflow.log_params({"prompt_name": PROMPT_NAME, "prompt_version": registered[v], "label": v,
                           "provider": PROVIDER, "model": MODEL, "temperature": TEMPERATURE,
                           "pricing_source": PRICING_SOURCE, "pricing_date": PRICING_DATE, "currency": CURRENCY})
        mlflow.log_metrics(m)
        mlflow.log_table(df.drop(columns=["response"]), "per_question_results.json")
        mlflow.log_text("\n\n".join(f"{r.question_id}: {r.response}" for r in df.itertuples()), "responses.txt")
    results.append({"prompt_version": v, **m})

df_all = pd.concat(all_rows)
df_all.to_csv("prompt_evaluation_results.csv", index=False)
comparison = pd.DataFrame(results)   # keep full precision
def style_comparison(d):
    return d.style.format({"avg_latency_seconds": "{:.3f}", "avg_input_tokens": "{:.1f}", "avg_output_tokens": "{:.1f}",
                           "avg_cost_per_request": "{:.8f}", "valid_json": "{:.2f}"})
display(style_comparison(comparison))
assert (df_all.status == "success").all(), "Some calls failed - do NOT promote; fix and rerun this cell."
```

	prompt_version	avg_latency_seconds	avg_input_tokens	avg_output_tokens	avg_cost_per_request	valid_json
0	v1	16.909	30.6	500.4	0.00002063	0.00
1	v2	13.421	52.6	358.6	0.00001540	0.00
2	v3	4.485	86.6	115.2	0.00000634	1.00

```
# Promotion rule: highest JSON validity, then lowest cost. Read the answers in the CSV before accepting!
assert (df_all.status == "success").all(), "Some calls failed; fix before promoting."
best = comparison.sort_values(["valid_json", "avg_cost_per_request"], ascending=[False, True]).iloc[0]["prompt_version"]
mlflow.genai.set_prompt_alias(PROMPT_NAME, alias="production", version=registered[best])
print("Promoted", best, "(registry version", registered[best], ") to alias 'production'")
print(mlflow.genai.load_prompt(f"prompts:/{PROMPT_NAME}@production").template[:120], "...")
comparison["recommended"] = comparison.prompt_version.eq(best).map({True: "Yes", False: "No"})
display(style_comparison(comparison))
```

```
Promoted v3 (registry version 3 ) to alias 'production'
You are an academic assistant.
Answer the student question and return ONLY valid JSON.
Do not include markdown, explanat ...
```

	prompt_version	avg_latency_seconds	avg_input_tokens	avg_output_tokens	avg_cost_per_request	valid_json	recommended
0	v1	16.909	30.6	500.4	0.00002063	0.00	No
1	v2	13.421	52.6	358.6	0.00001540	0.00	No
2	v3	4.485	86.6	115.2	0.00000634	1.00	Yes

Promotion justification (write 150+ words using your numbers)

Cover response quality (read the answers in `prompt_evaluation_results.csv`), JSON validity rate, latency, input/output tokens and cost per request. Explain the trade-off: does v3's JSON guarantee justify any extra tokens/cost compared with v1/v2?

Screenshots to capture from the MLflow UI: experiment runs + metrics, Prompt Registry (v1-v3), prompt compare view, `production` alias.

```
# Promotion justification: paragraphs 1, 2, 4 and 5 are generated from the measured results; paragraph 3 (answer quality) i
from collections import Counter

def parse_loose(text):
    t = str(text).strip()
    if t.startswith("```"):
        t = t.strip("`")
        t = t[4:] if t.lower().startswith("json") else t
    try:
        return json.loads(t)
    except Exception:
        return None

c = comparison.set_index("prompt_version")
ratio = OUTPUT_PRICE_PER_M / INPUT_PRICE_PER_M
v3rows = df_all[df_all.prompt_version == "v3"]
parsed = [p for p in (parse_loose(x) for x in v3rows.response) if isinstance(p, dict)]
n = max(len(parsed), 1)
kp = sum(len(p.get("key_points", [])) for p in parsed) / n
ans_words = sum(len(str(p.get("answer", "")).split()) for p in parsed) / n
conf_txt = ", ".join(f"{k} {m}" for k, m in Counter(str(p.get("confidence")) for p in parsed).items()) or "none"
w = {v: df_all[df_all.prompt_version == v].response.str.split().str.len().mean() for v in ["v1", "v2"]}
fewest, cheapest, fastest = c.avg_output_tokens.idxmin(), c.avg_cost_per_request.idxmin(), c.avg_latency_seconds.idxmin()
cost = c.avg_cost_per_request
cmin_free = min(cost["v1"], cost["v2"])
extra_in = c.avg_input_tokens["v3"] - c.avg_input_tokens["v2"]

p1 = (f"Prompt {best} (registry version {registered[best]}) was promoted to the production alias because the promotion rule
      f"ranks the valid_json rate first and average cost second. Valid JSON rates were {c.valid_json['v1']:.0%} for v1, "
      f"{c.valid_json['v2']:.0%} for v2 and {c.valid_json['v3']:.0%} for v3. This matters because downstream code must "
      f"parse every response.")

if fewest == cheapest == fastest:
    why = (f" Version {fewest} generated the fewest output tokens and was also the fastest and cheapest, because output "
           f"tokens are priced {ratio:.1f} times higher than input tokens and generation time grows with output length.")
else:
    why = " The fastest, cheapest and shortest versions differ, so the ranking depends on how each metric is weighted."
p2 = (f"Average latency was {c.avg_latency_seconds['v1']:.2f} s for v1, {c.avg_latency_seconds['v2']:.2f} s for v2 and "
      f"{c.avg_latency_seconds['v3']:.2f} s for v3. Average input tokens were {c.avg_input_tokens['v1']:.0f}, "
      f"{c.avg_input_tokens['v2']:.0f} and {c.avg_input_tokens['v3']:.0f}, and average output tokens were "
      f"{c.avg_output_tokens['v1']:.0f}, {c.avg_output_tokens['v2']:.0f} and {c.avg_output_tokens['v3']:.0f}. Average cost
      f"request was {cost['v1']:.8f}, {cost['v2']:.8f} and {cost['v3']:.8f} {CURRENCY}." + why)

# Quality paragraph written by hand after reading all 15 answers (ratings in answer_quality_check.csv).
p3 = ("On quality, I read all 15 answers and rated each for correctness (answer_quality_check.csv). v1 "
      "wrongly says in Q4 that threads share the same stack and program counter. v2 was the most accurate "
      "(4 correct, 1 partly correct). v3 was correct on 3 of 5 but shallower (44-word answers, 3.0 key "
      "points), and its Q5 says the goal is \"low bias and low variance\", which overstates what the tradeoff "
      "allows. All five v3 answers say confidence \"high\", so that field is not calibrated. I accept this "
      "small loss of depth for output that always parses.")

if cost["v3"] <= cmin_free:
    p4 = (f"The JSON guarantee did not cost extra here: v3 added {extra_in:.0f} input tokens per request compared with v2,
          f"but its shorter outputs made it cheaper, so the structured format came with lower cost and latency.")
else:
    pct = (cost["v3"] / cmin_free - 1) * 100
    p4 = (f"The JSON guarantee cost {pct:.0f}% more per request than the cheapest free-text prompt, about "
          f"{(cost['v3'] - cmin_free) * MONTHLY_REQUESTS:.4f} {CURRENCY} per {MONTHLY_REQUESTS:,} requests, a small price f
p5 = ("A valid JSON rate measured on five samples does not prove reliability at scale, so validity should be monitored, "
```

```
      "keeping v1 and v2 available for rollback through aliases.")

PROMO_TEXT = "\n\n".join([p1, p2, p3, p4, p5])
PROMO_WORDS = len(PROMO_TEXT.split())
assert PROMO_WORDS >= 150, f"Justification has only {PROMO_WORDS} words; the brief needs at least 150."
open("promotion_justification.txt", "w").write(PROMO_TEXT)
print(f"Promotion justification: {PROMO_WORDS} words (saved to promotion_justification.txt)")
display(Markdown(PROMO_TEXT.replace("\n\n", "\n\n")))
```

```
Promotion justification: 279 words (saved to promotion_justification.txt)
```

Prompt v3 (registry version 3) was promoted to the production alias because the promotion rule ranks the valid_json rate first and average cost second. Valid JSON rates were 0% for v1, 0% for v2 and 100% for v3. This matters because downstream code must parse every response.

Average latency was 16.91 s for v1, 13.42 s for v2 and 4.48 s for v3. Average input tokens were 31, 53 and 87, and average output tokens were 500, 359 and 115. Average cost per request was 0.00002063, 0.00001540 and 0.00000634 USD. Version v3 generated the fewest output tokens and was also the fastest and cheapest, because output tokens are priced 2.0 times higher than input tokens and generation time grows with output length.

On quality, I read all 15 answers and rated each for correctness (answer_quality_check.csv). v1 wrongly says in Q4 that threads share the same stack and program counter. v2 was the most accurate (4 correct, 1 partly correct). v3 was correct on 3 of 5 but shallower (44-word answers, 3.0 key points), and its Q5 says the goal is "low bias and low variance", which overstates what the tradeoff allows. All five v3 answers say confidence "high", so that field is not calibrated. I accept this small loss of depth for output that always parses.

The JSON guarantee did not cost extra here: v3 added 34 input tokens per request compared with v2, but its shorter outputs made it cheaper, so the structured format came with lower cost and latency.

A valid JSON rate measured on five samples does not prove reliability at scale, so validity should be monitored, keeping v1 and v2 available for rollback through aliases.

Generate report, README, requirements and download (run last)

Note on `report.pdf`

The cell below generates a first draft of `report.pdf`. The submitted `report.pdf` is a revised version of that draft: it was edited after the manual quality review (answer-quality findings and evidence links added). Every figure in it comes from the same CSVs and MLflow runs shown in this notebook. Re-running this cell would overwrite the submitted report with the draft.

```
import shutil, glob
from importlib.metadata import version as pkg_version

subprocess.run([sys.executable, "-m", "pip", "-q", "install", "reportlab"], check=True)
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import getSampleStyleSheet
from reportlab.lib import colors
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle
from xml.sax.saxutils import escape

PYV = sys.version.split()[0]
LIBS = {p: pkg_version(p) for p in ["mlflow", "openai", "pandas", "reportlab"]}

# ---------- requirements.txt ----------
open("requirements.txt", "w").write("\n".join(f"{k}=={v}" for k, v in LIBS.items()) + "\n")

# ---------- report text (500-700 words) ----------
lat = df_t1["latency_seconds"]
c = comparison.set_index("prompt_version")
sections = [
 ("1. Introduction",
  "MLOps manages datasets, training, deployment and monitoring of models. LLMOps extends this to LLM applications, where "
  "prompts, model versions and output guardrails are versioned artifacts and every request has a token-based cost and a lat
  "This project measures both for EduAssist AI's Q&A assistant, then promotes the best of three prompt versions in MLflow."
 ("2. Setup and deviation from the brief",
  f"OpenAI API access was unavailable (calls failed with an insufficient_quota error), so, as section 7 of the brief allows
  f"the project uses a locally hosted open-source model: {MODEL}, served by Ollama on Google Colab ({GPU_NAME}) and called
  f"through its OpenAI-compatible chat completions endpoint with the openai SDK, so no API key is needed. Temperature was '
  f"{TEMPERATURE}; token counts come from each response's usage field. Local inference has no bill, so cost is estimated fr
  f"at {INPUT_PRICE_PER_M} {CURRENCY} per million input tokens and {OUTPUT_PRICE_PER_M} {CURRENCY} per million output token
 ("3. Task 1: latency",
  f"Prompt v2, five questions. Latency in seconds: mean {lat.mean():.2f}, median {lat.median():.2f}, "
  f"minimum {lat.min():.2f}, maximum {lat.max():.2f}, standard deviation {lat.std(ddof=1):.2f}. " + INTERP_T1),
 ("4. Task 2: tokens and cost",
  f"Average cost was {avg_cost:.8f} {CURRENCY} per request; projected monthly cost for {MONTHLY_REQUESTS:,} requests "
  f"is {monthly:.2f} {CURRENCY}. " + REC_T2),
 ("5. Task 3: prompt versions in MLflow",
  f"Experiment {EXPERIMENT_NAME} holds one run per version; prompts are registered as {PROMPT_NAME} versions 1 to 3 with cc
]
TABLE = [["Version", "Avg latency (s)", "Avg input tok", "Avg output tok", "Avg cost (" + CURRENCY + ")", "Valid JSON", "Re
for v in ["v1", "v2", "v3"]:
    TABLE.append([v, f"{c.avg_latency_seconds[v]:.2f}", f"{c.avg_input_tokens[v]:.1f}", f"{c.avg_output_tokens[v]:.1f}",
                  f"{c.avg_cost_per_request[v]:.8f}", f"{c.valid_json[v]:.0%}", c.recommended[v]])
tail = ("6. Promotion justification", PROMO_TEXT)
```

```
body_words = sum(len((h + " " + t).split()) for h, t in sections + [tail])
table_words = sum(len(cell.split()) for row in TABLE for cell in row)
REPORT_WORDS = body_words + table_words
print(f"Report length: {REPORT_WORDS} words (brief asks for 500-700)")
if not 500 <= REPORT_WORDS <= 700:
    print("WARNING: outside 500-700 words. Shorten or lengthen the sections list above and rerun this cell.")

ss = getSampleStyleSheet()
doc = SimpleDocTemplate("report.pdf", pagesize=A4, leftMargin=48, rightMargin=48, topMargin=48, bottomMargin=48,
                        title="LLMOps Assignment 2 - EduAssist AI report")
story = [Paragraph("LLMOps Assignment 2: API Cost, Latency and Prompt Version Management", ss["Title"]),
         Paragraph("EduAssist AI academic Q&amp;amp;A assistant", ss["Normal"]), Spacer(1, 10)]
for h, t in sections:
    story += [Paragraph(escape(h), ss["Heading3"]), Paragraph(escape(t), ss["BodyText"])]
tbl = Table(TABLE, repeatRows=1)
tbl.setStyle(TableStyle([("BACKGROUND", (0, 0), (-1, 0), colors.lightgrey), ("GRID", (0, 0), (-1, -1), 0.5, colors.grey),
                         ("FONTSIZE", (0, 0), (-1, -1), 8), ("VALIGN", (0, 0), (-1, -1), "MIDDLE")]))
story += [Spacer(1, 6), tbl, Spacer(1, 8), Paragraph(escape(tail[0]), ss["Heading3"])]
for para in tail[1].split("\n\n"):
    story.append(Paragraph(escape(para), ss["BodyText"]))
doc.build(story)

# ---------- README.md ----------
README = f"""# LLMOps Assignment 2: API Cost, Latency and Prompt Version Management

Scenario: EduAssist AI academic Q&A assistant. Model: `{MODEL}` served by Ollama on Google Colab ({GPU_NAME}).
OpenAI access was unavailable (insufficient_quota), so a locally hosted open-source model is used as allowed in section 7 c
Cost is estimated from {PRICING_SOURCE} (checked {PRICING_DATE}, currency {CURRENCY}).

## Environment
- Python: {PYV}
- Libraries (also in requirements.txt): {", ".join(f"{k}=={v}" for k, v in LIBS.items())}
- Ollama (installed with `curl -fsSL https://ollama.com/install.sh | sh`), model `{MODEL}`
- Environment variables: none are required for the local model. If you switch back to OpenAI, set `OPENAI_API_KEY` in the
  shell that starts Jupyter and never put the key in the notebook.

## Install
```
pip install -r requirements.txt
curl -fsSL https://ollama.com/install.sh | sh
ollama serve &
ollama pull {MODEL}
```

## Reproduce the experiment
1. Open the notebook (Google Colab with a T4 GPU is recommended; the same code runs locally if Ollama is running).
2. Fill PRICING_SOURCE, PRICING_DATE and both prices in the configuration cell, then set PRICES_VERIFIED = True.
3. Run all cells top to bottom. This calls the model for 5 questions with v2 (Tasks 1-2) and for 5 questions with each of v
   logs one MLflow run per version, and sets the `production` alias.
4. Outputs: `latency_tokens_cost.csv`, `prompt_evaluation_results.csv`, `promotion_justification.txt`, `report.pdf`, `mlflo

## MLflow UI
Run this in the folder that contains `mlflow.db`, then open http://127.0.0.1:5000
```
mlflow server --host 127.0.0.1 --port 5000 --workers 1 --backend-store-uri sqlite:///mlflow.db
```
Screenshots required: experiment runs with metrics, Prompt Registry (v1-v3), prompt diff/compare view, `production` alias.

## Files
notebook (.ipynb) and its PDF/HTML export, requirements.txt, latency_tokens_cost.csv, prompt_evaluation_results.csv,
report.pdf, MLflow screenshots, this README.
"""
open("README.md", "w").write(README)

# ---------- bundle everything for download ----------
shutil.rmtree("submission", ignore_errors=True)
os.makedirs("submission")
for f in ["latency_tokens_cost.csv", "prompt_evaluation_results.csv", "promotion_justification.txt", "report.pdf",
          "README.md", "requirements.txt", "mlflow.db"]:
    shutil.copy(f, "submission")
for d in ["mlruns", "mlartifacts"]:
    if os.path.isdir(d):
        shutil.copytree(d, os.path.join("submission", d))
shutil.make_archive("submission_files", "zip", "submission")
print("Created submission_files.zip with:", sorted(os.listdir("submission")))

try:
    from google.colab import files
    files.download("submission_files.zip")
except ImportError:
    pass
```

```
print("""
Next: take the 5 MLflow screenshots on your laptop (see README), then File > Download .ipynb and File > Print (save as PDF)
and put everything into one ZIP: notebook, notebook PDF/HTML, the files in submission_files.zip, and the screenshots.""")
```

```
Report length: 672 words (brief asks for 500-700)
Created submission_files.zip with: ['README.md', 'latency_tokens_cost.csv', 'mlflow.db', 'mlruns', 'promotion_justification.

Next: take the 5 MLflow screenshots on your laptop (see README), then File > Download .ipynb and File > Print (save as PDF),
and put everything into one ZIP: notebook, notebook PDF/HTML, the files in submission_files.zip, and the screenshots.
```